

TaskFlow API

How to run it & test every endpoint — a hands-on guide

For someone who just opened Postman and didn't know what to paste.

No prior API-testing experience assumed. Three ways shown: Swagger (easiest), Postman, curl.

Spring Boot 3 · runs at `http://localhost:8080` · Generated 4 June 2026

THE 30-SECOND MENTAL MODEL

Your API is a program waiting at the address `http://localhost:8080` ("localhost" = *this computer*; "8080" = the door number it listens on). You test it by sending it **HTTP requests** and reading the **responses**. Almost every endpoint is **locked** 🔒 — you must first **register/login to get a token**, then attach that token to every other request. That token step is the bit Postman didn't make obvious. This guide fixes that.

Contents

1. Step 0 — Get the app running (must do first)
2. Step 1 — The URLs you'll paste
3. Step 2 — The golden rule: get a token, then attach it
4. Method A — Swagger UI (easiest, no Postman needed)
5. Method B — Postman, click by click
6. Method C — curl (command line)
7. Full endpoint reference (every URL + example body)
8. Understanding responses & fixing errors
9. Troubleshooting checklist

Step 0 — Get the app running (must do first)

Nothing can be tested until two things are running: the **database** and the **app**.

1 Start Docker Desktop (the whale icon). Wait until it says "running."

2 Start the database — in a terminal inside the project folder:

```
docker compose up -d postgres
```

This starts PostgreSQL in the background on port **5433**. Check it's healthy with `docker ps`.

3 Start the app:

```
./mvnw spring-boot:run      # macOS/Linux/Git-Bash  
mvnw spring-boot:run      # Windows cmd
```

Wait for a line like `Started TaskflowApplication in 4.2 seconds`. The app is now listening.

4 Confirm it's alive — open this in your browser:

```
http://localhost:8080/actuator/health
```

You should see `{"status": "UP"}`. If you do, you're ready to test. 🎉

WATCH OUT

Leave the app running in its terminal window while you test. Closing that window (or pressing `Ctrl+C`) stops the app, and every request will then fail with "connection refused."

Step 1 — The URLs you'll paste

Every address starts with the **base URL** and the business endpoints sit under `/api/v1` :

Thing	URL
Base URL	<code>http://localhost:8080</code>
All API endpoints start with	<code>http://localhost:8080/api/v1</code>
Swagger UI (interactive tester)	<code>http://localhost:8080/swagger-ui.html</code>
Health check	<code>http://localhost:8080/actuator/health</code>
Raw API spec (JSON)	<code>http://localhost:8080/v3/api-docs</code>

So to create a task, the full URL you paste into Postman is:

`http://localhost:8080/api/v1/tasks`

Step 2 — The golden rule: get a token, then attach it

WHY MOST REQUESTS “DON’T WORK” AT FIRST

Only two endpoints are **OPEN** 🗝️ — `register` and `login`. Everything else is **LOCKED** 🗝️. To open the locked ones you must:

1. **Register** (or **login**). The response contains a long `"token"` string (a JWT).
2. **Copy that token**.
3. **Attach it** to every other request as a header: `Authorization: Bearer <token>`.

That’s it. The token is your “I’m logged in” wristband; you flash it at the door each time.

A token looks like this (three chunks separated by dots) — yours will differ:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ6dWV7sD5P6HB6qih70dTZ...
```

GOOD NEWS — A READY-MADE TEST ACCOUNT

The dev database is pre-seeded with a demo user, so you can log in immediately without registering:

Email: `demo@taskflow.com` · **Password:** `Password123!`

It already owns two categories and five tasks — handy for trying the `GET` endpoints right away.

Method A — Swagger UI (easiest, no Postman needed)

Your app generates a clickable test page for itself. Start here — it's the gentlest way to learn.

1 Open `http://localhost:8080/swagger-ui.html` in your browser. You'll see every endpoint grouped by feature (auth, user, category, task), each expandable.

2 **Log in to get a token.** Expand `POST /api/v1/auth/login` → click **"Try it out"** → in the body box type:

```
{ "email": "demo@taskflow.com", "password": "Password123!" }
```

→ click **Execute**. Scroll to the response and copy the value of `"token"` (the long string, without the surrounding quotes).

3 **Tell Swagger to use the token.** Click the green **Authorize**  button near the top-right. Paste your token into the box and click **Authorize**, then **Close**. (Depending on the version, the box may already prepend `"Bearer"` — if it asks for the value only, just paste the token.)

4 **Now test anything.** Every locked endpoint will work because Swagger attaches your token automatically. Try `GET /api/v1/tasks` → Try it out → Execute → you'll see the demo user's tasks.

TIP

Swagger also *documents* the API: it shows exactly which fields each request accepts and what each response looks like. When you forget a body shape, look it up here rather than guessing.

Method B — Postman, click by click

Postman is a standalone app for sending requests. Here's the exact sequence you were missing.

B1 · Register or log in (an OPEN request)

1 Click + to open a new request tab.

2 Set the method dropdown (left of the URL bar) to **POST**.

3 In the URL bar, paste:

```
http://localhost:8080/api/v1/auth/login
```

4 Click the **Body** tab (under the URL bar) → choose **raw** → in the dropdown on the right (it says "Text") choose **JSON**. **This step is critical** — it tells the server you're sending JSON.

5 Paste the body:

```
{ "email": "demo@taskflow.com", "password": "Password123!" }
```

6 Click **Send**. In the response pane (bottom) you'll get **200** and a JSON body. **Copy the token value**.

THE #1 POSTMAN MISTAKE

If you skip step 4 (leaving Body as "Text" instead of "JSON"), the server doesn't recognise your JSON and you'll get a **415** or **400** error. Always set **raw** → **JSON** for POST/PUT requests.

B2 · Call a LOCKED endpoint (attach the token)

1 New request → method **GET** → URL

```
http://localhost:8080/api/v1/tasks
```

2 Click the **Authorization** tab → **Type** dropdown → choose **Bearer Token**.

3 Paste **just the token** into the "Token" box on the right. (Postman adds the word **Bearer** for you — don't type it yourself here.)

4 Click **Send**. You should get `200` and a list of tasks. You're in. 🎉

PRO TIP — STOP COPY-PASTING THE TOKEN EVERY TIME

In Postman, create an **Environment** with a variable called `token`. Paste your token as its value once. Then in any request's Authorization → Bearer Token box, type `{{token}}`. Now every request reuses it. (Advanced: a small "Tests" script on the login request can capture the token automatically — ask me and I'll give you the snippet.)

B3 · Creating things (POST/PUT with a body + a token)

For `POST /api/v1/tasks` you need **both** a JSON body (B1, step 4–5) **and** the Bearer token (B2, step 2–3). Example body:

```
{ "title": "Buy milk", "priority": "LOW", "dueDate": "2026-07-01T09:00:00Z" }
```

Method C — curl (command line)

If you prefer the terminal, `curl` sends the same requests. Register/login first, copy the token, then pass it with `-H "Authorization: Bearer ..."`.

```
# 1) Log in and see the token
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"demo@taskflow.com","password":"Password123!"}'

# 2) Use the token (paste it in place of THE_TOKEN)
curl http://localhost:8080/api/v1/tasks \
  -H "Authorization: Bearer THE_TOKEN"

# 3) Create a task
curl -X POST http://localhost:8080/api/v1/tasks \
  -H "Authorization: Bearer THE_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"title":"Buy milk","priority":"LOW"}'
```

On Windows `cmd`, use `^` instead of `\` for line breaks, or just put the whole command on one line.

Full endpoint reference

Every endpoint in the app today. 🔓 = open (no token). 🔒 = needs a Bearer token.

Auth — getting in

POST /api/v1/auth/register

🔓 OPEN

Create a new account. Returns **201** with a token + your user info.

Body:

```
{ "email": "you@example.com", "password": "Password123", "displayName": "Your Name" }
```

Rules: valid email; password 8–72 chars; displayName required. Duplicate email → **409**.

POST /api/v1/auth/login

🔓 OPEN

Exchange email + password for a token. Returns **200**.

```
{ "email": "you@example.com", "password": "Password123" }
```

Response shape (register & login):

```
{ "token": "eyJ...", "tokenType": "Bearer",  
  "user": { "id": "...", "email": "...", "displayName": "...", "createdAt": "..." } }
```

User

GET /api/v1/users/me

🔒 token

Returns *your own* profile (decided by the token, not by any parameter). **200**.

Categories

GET /api/v1/categories

🔒 token

List *your* categories. **200** → an array of `{ id, name, color }`.

POST /api/v1/categories

 token

Create a category. **201** .

```
{ "name": "Work", "color": "#3B82F6" }
```

name required (≤ 100 chars); color optional (≤ 20 chars). There is no "owner" field — it's taken from your token.

GET /api/v1/categories/{id}

 token

Get one of your categories by its id. Not yours / not found → **404** .

PUT /api/v1/categories/{id}

 token

Replace a category's fields. **200** . Same body shape as create.

DELETE /api/v1/categories/{id}

 token

Delete a category. **204** (empty body). Its tasks survive, just uncategorised.

Tasks

GET /api/v1/tasks

 token

List *your* tasks. **200** → an array of task objects (each with its category nested, or null).

POST /api/v1/tasks

🔒 token

Create a task. 201 .

```
{
  "title": "Write report",          // required (≤255 chars)
  "description": "Q3 numbers",      // optional
  "status": "TODO",                // optional: TODO | IN_PROGRESS | DONE (default TODO)
  "priority": "HIGH",              // optional: LOW | MEDIUM | HIGH (default MEDIUM)
  "dueDate": "2026-07-01T09:00:00Z", // optional, ISO-8601 UTC instant
  "categoryId": "575f2a01-..."    // optional; must be one of YOUR categories
}
```

Smallest valid body: { "title": "Anything" }. An unknown status/priority value → 400 .

GET /api/v1/tasks/{id}

🔒 token

Get one of your tasks. Not yours / not found → 404 .

PUT /api/v1/tasks/{id}

🔒 token

Replace a task's fields. 200 . Same body shape as create.

PATCH /api/v1/tasks/{id}/complete

🔒 token

Toggles done-state. No body needed. If the task isn't DONE it becomes DONE (and stamps `completedAt`); if it is DONE it flips back to TODO (and clears `completedAt`). 200 .

DELETE /api/v1/tasks/{id}

🔒 token

Delete a task. 204 (empty body).

Understanding responses & fixing errors

The **status code** tells you what happened. 2xx = good; 4xx = something about *your request*; 5xx = the server broke.

Code	Meaning	Typical cause / fix
200 OK	Success (read/update).	All good.
201 Created	A new thing was created.	Returned by register, POST category, POST task.
204 No Content	Success, nothing to return.	Returned by DELETE. Empty body is expected.
400 Bad Request	Your input broke a rule.	Missing <code>title</code> , password too short, bad enum value, or a <code>categoryId</code> that isn't yours.
401 Unauthorized	No valid token.	You forgot the <code>Authorization</code> header, or the token is wrong/expired (tokens last 24h). Log in again.
403 Forbidden	Authenticated but not allowed.	Rare here; usually means a config/path issue.
404 Not Found	No such resource (or not yours).	Wrong id, or you're asking for another user's item — by design it looks identical to "doesn't exist."
409 Conflict	Clashes with existing data.	Registering an email that already exists.
415 Unsupported Media Type	Body wasn't sent as JSON.	In Postman set Body → raw → JSON . In curl add <code>-H "Content-Type: application/json"</code> .
500 Server Error	The app threw an unexpected error.	Check the app's terminal log for the stack trace.

CONCEPT — THE TOKEN EXPIRES

Tokens are valid for **24 hours**. If calls that worked yesterday suddenly return `401`, your token expired — just log in again and use the new one.

Troubleshooting checklist

Symptom	Most likely cause & fix
"Could not get response / Connection refused"	The app isn't running, or you used the wrong port. Restart <code>./mvnw spring-boot:run</code> and confirm <code>http://localhost:8080/actuator/health</code> shows <code>{"status": "UP"}</code> .
Everything returns 401	You didn't attach the token, attached it wrong (typed "Bearer" twice), or it expired. Re-login; in Postman use Authorization → Bearer Token and paste only the token.
400 on POST/PUT	A validation rule failed (missing <code>title</code> , short password) or the body wasn't JSON. Check the message in the response and that Body = raw → JSON.
App won't start: "relation ... does not exist" or migration error	The database container isn't up or isn't migrated. Run <code>docker compose up -d postgres</code> and restart the app so Flyway runs.
App won't start: port 8080 already in use	An old run is still alive. Stop it (close its terminal) or kill whatever holds port 8080, then start again.
Login fails for the demo user	Make sure you're on the dev profile (the default) — only dev seeds the demo data. Password is <code>Password123!</code> (with the "!").
Can't reach Swagger	Use exactly <code>http://localhost:8080/swagger-ui.html</code> and make sure the app is running.

TaskFlow API testing guide · base URL `http://localhost:8080` · endpoints under `/api/v1` · reflects the app after Phase 7. New endpoints (filtering, analytics) will be added in later phases — re-check Swagger, which always shows the current, live list.